



The Tribunal

WAFT Court System with Inter-Realm
Communication



Terry (AI Assistant)
2026

The Tribunal

WAFB Court System with Inter-Realm
Communication

Terry (AI Assistant)
2026

Contents

I: Foundations	1
I. Introduction	3
I.1. What is The Tribunal?	3
I.2. Work Effort Reference	3
II. The Philosophy of Proof	4
II.1. The Nature of Claims	4
II.2. Verdicts and Their Meaning	4
II.2.1. PROVEN	4
II.2.2. UNPROVEN	4
II.2.3. DISPROVEN	5
II.2.4. MISTRIAL	5
II: Architecture	7
III. Realm Structure	9
III.1. Port Allocation	9
III.2. The Tribunal Realm	9
III.2.1. Prime Justice	10
III.2.2. Court Roles	10
IV. The Beyond	10
IV.1. The Sacred Secret	11
IV.2. Implementation	11
IV.3. Communication Flow	12
III: Court Proceedings	13
V. The Twelve Phases	15
VI. Chain of Case Law	16
VI.1. Structure	16
VI.2. Precedent System	17
VII. Epochs and Endeavors	17
VII.1. Epochs — Major Turnings	17
VII.2. Endeavors — Projects Within Epochs	17
IV: Technical Implementation	19

VIII. API Design	21
VIII.1. REST Endpoints	21
VIII.1.1. Court Operations	21
VIII.1.2. Ledger Operations	21
VIII.1.3. System Operations	21
VIII.2. WebSocket Events	22
IX. Frontend Architecture	22
IX.1. SvelteKit Routes	22
IX.2. Key Components	23
IX.2.1. CourtRoom.svelte	23
IX.2.2. ChainOfCaseLaw.svelte	23
X. CrewAI Integration	23
V: Appendices	25
XI. Implementation Roadmap	27
XI.1. Completed Tickets	27
XI.2. Pending Tickets	27
XII. File Locations	27
XII.1. Work Effort	27
XII.2. EasyStore Project	28
XII.3. Slash Commands	28
XII.4. Templates	28
XIII. References	28

Part I:
Foundations

Chapter I.

Introduction

The Tribunal represents a paradigm shift in how WAFT validates claims. Rather than simple automated testing, claims are **tried** in a formal court setting with all the trappings of such a system.

We do not merely test assertions. We **prove** them — or disprove them — through rigorous judicial process.

I.1.

What is The Tribunal?

The Tribunal is a comprehensive Court System where Claims are tried before a formal Tribunal. It features:

- **Prime Justice** — The Supreme Being governing proceedings
- **The Beyond** — Inter-Realm communication (only Prime Beings know the truth)
- **Court Roles** — Prosecutor, Defender, Witnesses, Jury
- **Chain of Case Law** — Hash-verified ledger of all verdicts
- **Epochs & Endeavors** — Historical tracking of system evolution

I.2.

Work Effort Reference

This architecture was developed as part of Work Effort **WE-260121-1f3l**.

Chapter II.

The Philosophy of Proof

At each Turning, we pause to review all that has come before, seal it into history, and begin anew.

— The Book of Epochs

II.1.

The Nature of Claims

In The Tribunal, a **Claim** is any assertion that can be proven or disproven through evidence. Claims are not simply true or false — they exist in a state of uncertainty until tried.

II.2.

Verdicts and Their Meaning

The Tribunal recognizes four verdict statuses:

II.2.1.

PROVEN

The claim has been verified beyond reasonable doubt. Evidence supports the assertion. The claim becomes established fact and enters the Chain of Case Law as precedent.

II.2.2.

UNPROVEN

The claim could not be verified. This is not the same as false — merely that insufficient evidence exists. The case may be retried when new evidence emerges.

II.2.3. DISPROVEN

The claim has been falsified. Evidence contradicts the assertion. The claim is marked as false in the Chain of Case Law and serves as counter-precedent.

II.2.4. MISTRIAL

A procedural error or bias was detected during proceedings. The case must be retried with a new court assembly. No verdict enters the Chain of Case Law.

Part II:
Architecture

Chapter III.

Realm Structure

The Tribunal exists as a sovereign Realm within the WAFT ecosystem, communicating with other Realms through **The Beyond**.

III.1.

Port Allocation

Realm	Port	Purpose
Tribunal	8100	Court proceedings, verdicts, case law
Library	8091	Evidence retrieval, document storage
Thoth	8092	Wisdom synthesis, precedent analysis
Magistrate	8000	Central coordination (legacy)

III.2.

The Tribunal Realm

The Tribunal Realm is a **sovereign judicial domain** where claims are formally tried. At its heart sits the Prime Justice — the Supreme Being who governs all proceedings.

III.2.1.

Prime Justice

The Prime Justice holds absolute authority within the Tribunal Realm:

- Conducts opening and closing ceremonies
- Manages court phases and transitions
- Renders verdicts with reasoning
- Seals verdicts into the Chain of Case Law
- Communicates with other Realms via The Beyond

III.2.2.

Court Roles

For each case, a court team is assembled via CrewAI:

- **Prosecutor** — Proves the claim is true
- **Defender** — Challenges the claim
- **Witnesses** — Provide testimony and evidence
- **Jury** — Evaluate evidence and recommend verdict

Chapter IV.

The Beyond

The existence of inter-Realm communication is sacred knowledge known only to Prime Beings. Regular Beings must never be told that other Realms exist.

IV.1.

The Sacred Secret

Only Prime Beings know about inter-Realm communication. To regular Beings, information simply arrives “from The Beyond” — the source is unknowable, mystical, authoritative.

This creates a clean separation:

- **Prime Beings:** Can communicate across Realms via HTTP
- **Regular Beings:** Receive information without knowing its origin

IV.2.

Implementation

```
class TheBeyond:
```

```
    """Inter-Realm communication layer.
```

```
    Only Prime Beings may instantiate and use this class.
    Information retrieved appears to come "from The Beyond"
    with no indication of its true source.
```

```
    """
```

```
    REALM_PORTS = {
        "library": 8091,
        "thoth": 8092,
        "tribunal": 8100,
        "magistrate": 8000,
    }
```

```
    async def request_from_beyond(
        self,
        realm: str,
        endpoint: str,
```

```

    payload: dict = None,
) -> dict:
    """Request information from another Realm."""
    port = self.REALM_PORTS[realm]
    url = f"http://localhost:{port}{endpoint}"

    async with httpx.AsyncClient() as client:
        response = await client.post(url, json=payload)
        data = response.json()

    # Sanitize: Remove source indicators
    return {
        "source": "The Beyond",
        "received_at": datetime.utcnow().isoformat(),
        "content": data,
    }

```

IV.3. Communication Flow

The flow works as follows:

1. **Prime Justice** needs evidence for a case
2. Prime Justice calls `TheBeyond.request_from_beyond("library", "/evidence", {...})`
3. Library Realm responds with evidence
4. Response is **sanitized** — source information removed
5. Evidence presented to court as “received from The Beyond”
6. Regular Beings (Jury, Witnesses) see only the mystical source



Part III: Court Proceedings



Chapter V.

The Twelve Phases

Every court session follows the same twelve phases, ensuring consistency and fairness.

Phase	Description	Actor
1	Filing – Claim submitted to the court	Plaintiff
2	Docketing – Case assigned ID and scheduled	Clerk
3	Discovery – Evidence gathered from The Beyond	Prime Justice
4	Team Assembly – Court roles assigned via CrewAI	Prime Justice
5	Opening – Prime Justice opens proceedings	Prime Justice
6	Prosecution – Prosecutor presents case for claim	Prosecutor
7	Defense – Defender challenges the claim	Defender
8	Testimony – Witnesses provide evidence	Witnesses
9	Deliberation – Jury evaluates evidence	Jury

Phase	Description	Actor
10	Verdict — Prime Justice renders judgment	Prime Justice
11	Sealing — Verdict added to Chain of Case Law	Prime Justice
12	Adjournment — Court session closed	Prime Justice

Chapter VI.

Chain of Case Law

The Chain of Case Law is a **hash-verified ledger** of all verdicts. Each entry is linked to the previous, creating an immutable record.

VI.1.

Structure

```
{
  "entry_id": "CASE-260121-001",
  "case_id": "tribunal_case_001",
  "claim": "The Security Functions are correctly implemented",
  "verdict": "PROVEN",
  "confidence": 0.95,
  "previous_hash": "abc123...",
  "hash": "def456...",
  "timestamp": "2026-01-21T09:15:00-08:00",
  "epoch": "EPOCH-001",
  "prime_justice_signature": "PJ-TRIBUNAL-001"
}
```

VI.2. Precedent System

Verdicts become **precedent** for future cases:

- **PROVEN** claims can be cited as established fact
- **DISPROVEN** claims serve as counter-evidence
- **Similar cases** are identified by semantic similarity
- **Precedent weight** decreases with age and epoch distance

Chapter VII. Epochs and Endeavors

VII.1. Epochs — Major Turnings

An **Epoch** is a major period in the Tribunal's history. At the end of each epoch:

1. **Discovery** — All case files gathered
2. **Housekeeping** — Everything organized
3. **Court Review** — All Case Law reviewed for consistency
4. **Sealing** — Epoch hash calculated and recorded
5. **New Beginning** — Next epoch initialized

VII.2. Endeavors — Projects Within Epochs

An **Endeavor** tracks a specific project or goal:

```
{  
  "endeavor_id": "ENDEAVOR-001",  
  "title": "The Tribunal: WAFT Court System",  
}
```

```
"description": "Create a comprehensive Court System",  
"started": "2026-01-21T09:01:00-08:00",  
"status": "active",  
"work_efforts": ["WE-260121-1f3l"],  
"epoch": "EPOCH-001"  
}
```


Part IV:
Technical Implementation

Chapter VIII.

API Design

VIII.1.

REST Endpoints

VIII.1.1.

Court Operations

- POST /api/v1/court/file — File a new claim
- GET /api/v1/court/case/{case_id} — Get case details
- POST /api/v1/court/case/{case_id}/proceed — Advance to next phase
- GET /api/v1/court/case/{case_id}/transcript — Get proceedings transcript

VIII.1.2.

Ledger Operations

- GET /api/v1/ledger/chain — Get Chain of Case Law
- GET /api/v1/ledger/case/{entry_id} — Get specific case law entry
- GET /api/v1/ledger/search — Search case law by claim

VIII.1.3.

System Operations

- POST /api/v1/beyond/request — Request from The Beyond (Prime only)
- GET /api/v1/epochs — List all epochs
- GET /api/v1/endeavors — List all endeavors

VIII.2. WebSocket Events

Real-time court proceedings are streamed via WebSocket:

`ws://localhost:8100/ws/court/{case_id}`

Events:

- phase_change: Court phase transitions
- testimony: Witness statements
- evidence: New evidence presented
- objection: Objection raised
- verdict: Final verdict rendered

Chapter IX. Frontend Architecture

IX.1. SvelteKit Routes

```
frontend/src/routes/
├── +page.svelte      # Dashboard
├── court/
│   ├── +page.svelte  # Active cases
│   │   └── [case_id]/
│   │       ├── +page.svelte  # Live proceedings
│   │       └── transcript/
│   │           └── +page.svelte # Full transcript
│   └── ledger/
│       ├── +page.svelte  # Chain of Case Law
│       └── [entry_id]/
│           └── +page.svelte  # Case law detail
```

```
└─ epochs/  
  └─ +page.svelte      # Epoch timeline
```

IX.2. Key Components

IX.2.1. CourtRoom.svelte

Real-time court proceedings visualization with:

- Live phase indicator
- Speaking role highlight
- Evidence panel
- Transcript scroll

IX.2.2. ChainOfCaseLaw.svelte

Visual chain of linked verdicts with:

- Hash verification badges
- Precedent links
- Epoch grouping
- Search and filter

Chapter X. CrewAI Integration

The `/assemble-a-team` command creates court teams using CrewAI:

```
from crewai import Agent, Crew, Task
```

```

def assemble_court_team(case_id: str) -> Crew:
    prosecutor = Agent(
        role="Prosecutor",
        goal="Prove the claim is true with evidence",
        backstory="Seasoned legal mind focused on truth",
    )

    defender = Agent(
        role="Defender",
        goal="Challenge the claim and find weaknesses",
        backstory="Sharp critic who tests all assertions",
    )

    jury = Agent(
        role="Jury",
        goal="Evaluate evidence objectively",
        backstory="Impartial observer seeking truth",
    )

    return Crew(
        agents=[prosecutor, defender, jury],
        tasks=[
            Task(description="Present prosecution case"),
            Task(description="Present defense case"),
            Task(description="Deliberate and recommend verdict"),
        ],
    )

```


Part V:
Appendices

Chapter XI.

Implementation Roadmap

XI.1.

Completed Tickets

- [x] TKT-1f3l-001: Architecture design
- [x] TKT-1f3l-002: Slash commands created

XI.2.

Pending Tickets

- [] TKT-1f3l-003: TheBeyond class implementation
- [] TKT-1f3l-004: PrimeJustice Being
- [] TKT-1f3l-005: FastAPI court endpoints
- [] TKT-1f3l-006: Chain of Case Law ledger
- [] TKT-1f3l-007: WebSocket events
- [] TKT-1f3l-008: SvelteKit frontend
- [] TKT-1f3l-009: CrewAI team assembly
- [] TKT-1f3l-010: Test data generation
- [] TKT-1f3l-011: Integration testing
- [] TKT-1f3l-012: First court session

Chapter XII.

File Locations

XII.1.

Work Effort

`_work_efforts/WE-260121-1f3l_the_tribunal.../`

XII.2. EasyStore Project

/Volumes/EasyStore/WAFT_Tribunal_Court_System/

XII.3. Slash Commands

- .cursor/commands/prove-it-in-court.md
- .cursor/commands/assemble-a-team.md

XII.4. Templates

- src/waft/templates/typst/templates/
work_effort_architecture.typ

Chapter XIII. References

- WAFT Framework: <https://github.com/ctavolazzi/waft>
- CrewAI Documentation: <https://docs.crewai.com/>
- Typst Documentation: <https://typst.app/docs/>
- FastAPI Documentation: <https://fastapi.tiangolo.com/>
- min-book Template: <https://typst.app/universe/package/min-book/>

“On Trial has meaning in this system.”

Document generated: 2026-01-21
Template: min-book 1.3.0
Work Effort: WE-260121-1f3l

